

BD2 - Resumo para a Prova 2

Material de estudo baseado nos exercícios de laboratório (Oracle XE 10.2). Cada tópico segue a estrutura: **pra que serve**, **situação hipotética de uso** e **exemplo de script/query**.

Tópicos cobertos:

1. Views
 2. Database Links e Sinônimos
 3. Flash Back Queries
 4. Jobs de Replicação
 5. SQL*Loader
 6. Shell Script na Administração de BD
 7. Tuning de Consultas SQL
-

1. Views

Pra que serve

Uma view é uma consulta SQL armazenada que se comporta como uma tabela virtual. Ela não guarda dados, apenas a definição da consulta. Serve para dois propósitos principais:

- **Abstração e consistência:** esconde a complexidade de joins, renomeia colunas e isola a aplicação da estrutura física das tabelas. Se as tabelas de base mudarem (nome de campo, por exemplo), basta recriar a view sem alterar o código da aplicação.
- **Segurança da informação:** permite conceder acesso somente a alguns campos de uma tabela, sem expor a tabela inteira.

Situação hipotética

O usuário EOR precisa acessar dados de clientes da tabela TCLIENTES, que pertence ao usuário POS. Conceder `grant select on tclientes to eor` daria acesso a **todos** os campos (inclusive endereço e telefone), o que o diretor comercial não autoriza. A solução é criar uma view com apenas os campos permitidos (Código e Razão Social) e conceder acesso à view, não à tabela.

Exemplo de script/query

View para abstração (renomeando colunas e juntando tabelas):

```

create or replace view VCLI as
select cli.cod_cliente    CODCLI,
       cli.razao_social   RS,
       cli.data_cadastro  DTCAD,
       cid.descricao      CIDADE,
       cid.uf             UF
from cliente cli, cidade cid
where cli.cod_cidade = cid.cod_cidade
order by cli.razao_social;

-- o filtro usa o ALIAS da view (CODCLI), não o nome original da coluna
(COD_CLIENTE)
select * from vcli where CODCLI = 3499;

```

View para segurança + concessão de acesso:

```

conn pos/pos
create or replace view VCLIRS as
select cod_cli, descricao
from tclientes;

grant select on vclirs to eor;

conn eor/eor
select * from pos.vclirs;           -- funciona, qualificando pelo dono (POS)

-- para não precisar qualificar pelo nome do usuário, cria-se um sinônimo:
create synonym vclirs for pos.vclirs;
select * from vclirs;

```

Dica de prova: sem o `grant`, o acesso de `EOR` a `pos.vclirs` falha (sem privilégio). Sem o sinônimo, é obrigatório qualificar a view com o nome do dono (`pos.vclirs`).

2. Database Links e Sinônimos

Pra que serve

Um **database link** é um objeto que armazena uma conexão para outro banco de dados (remoto ou outro esquema do mesmo banco). Com ele, um usuário consulta e manipula tabelas de outro banco como se fossem locais, usando a sintaxe `tabela@link`. É a base do processo de **replicação de dados** no Oracle.

Um **sinônimo** é um apelido que oculta do usuário a localização física dos dados. Um sinônimo pode apontar para uma tabela remota (via database link), de forma que a aplicação nem perceba que os dados estão em outro servidor.

A conexão cliente-servidor Oracle usa por padrão a **porta 1521**, configurada no arquivo `tnsnames.ora`.

Situação hipotética

A `FILIAL1` (que não possui tabelas próprias) precisa consultar e inserir pedidos na tabela `PEDIDO` que está no banco `MATRIZ`. Cria-se um database link `DL1` apontando para a string de conexão da matriz (definida no `tnsnames.ora` como `BD1`) e acessa-se a tabela remota com `PEDIDO@DL1`. Se o host de destino não existir na rede (IP inalcançável), a consulta falha, o que pode ser diagnosticado com um simples `ping`.

Exemplo de script/query

```
-- string de conexão BD1 definida no tnsnames.ora aponta para
HOST/PORTA/SERVICE da matriz

conn filial1/filial1

create database link DL1
  connect to MATRIZ identified by MATRIZ
  using 'BD1';

-- consultar tabela remota
select * from pedido@dl1;

-- inserir registro remoto
insert into PEDIDO@DL1 (CODPED, CODCLI, CODPROD, QTDE, DATA, FILIAL)
values                ( 999,    10,    35,    50.8, sysdate, 1);
commit;

-- excluir registro remoto
delete from PEDIDO@DL1 where CODPED = 999;
commit;

-- sinônimo que oculta a localização física dos dados
create synonym PED for PEDIDO@DL1;

insert into PED (CODPED, CODCLI, CODPROD, QTDE, DATA, FILIAL)
values        ( 999,    10,    35,    50.8, sysdate, 1);
commit;
```

```
delete from PED where CODPED = 999;
commit;
```

Dica de prova: o erro de um database link costuma estar na string de conexão (tnsnames.ora): host inexistente, porta errada (ex.: 1525 em vez de 1521) ou IP fora da rede. Use `select * from user_tables;` para confirmar que um usuário não tem tabelas próprias.

3. Flash Back Queries

Pra que serve

Permite consultar o estado **passado** dos dados de uma tabela ("voltar no tempo"), usando a cláusula `AS OF TIMESTAMP`. O Oracle usa os dados de undo para reconstruir como a tabela estava em um determinado instante. Serve para recuperar dados alterados ou excluídos por engano **sem precisar restaurar um backup**.

Situação hipotética

Após inserir 3 produtos na tabela `ESTOQUE`, um usuário roda updates que zeram e alteram quantidades e, em seguida, faz um `delete from ESTOQUE` que apaga tudo. É preciso restaurar a tabela com a posição **imediatamente após a inserção dos 3 produtos** (produtos 1, 2 e 3 com quantidades 10, 20 e 30), usando um instante de tempo conhecido.

Exemplo de script/query

```
-- consultar como a tabela estava em um instante passado
select *
from estoque
as of timestamp to_timestamp('2023-05-18 10:27:00','YYYY-MM-DD HH24:MI:SS');

-- restaurar o valor de UMA coluna ao estado passado
update estoque
set qtde = ( select qtde
             from estoque
             as of timestamp to_timestamp('2023-05-18 10:27:00','YYYY-MM-DD
HH24:MI:SS')
           where cod = 3 )
where cod = 3;
commit;

-- restaurar a TABELA INTEIRA depois de um delete
```

```
insert into ESTOQUE
select *
from estoque
as of timestamp to_timestamp('2023-05-18 10:27:00','YYYY-MM-DD HH24:MI:SS');
commit;
```

Dica de prova: o segredo é escolher o timestamp logo **após** a operação que você quer preservar e **antes** da operação que estragou os dados. O Flash Back funciona enquanto os dados de undo ainda estiverem retidos.

4. Jobs de Replicação (DBMS_JOB)

Pra que serve

Jobs automatizam a execução **programada** de tarefas dentro do banco (procedures, refresh de snapshots, etc.), uma única vez ou de forma recorrente (a cada X minutos, diariamente, semanalmente). No contexto de replicação, são usados para manter um **snapshot** (materialized view) sincronizado com a tabela de origem.

Situação hipotética

A `FILIAL1` tem um snapshot da tabela `PEDIDO` cuja origem está em `MATRIZ`. Quer-se programar a atualização desse snapshot **a cada 3 minutos**, trazendo somente as alterações feitas na tabela original (refresh do tipo `'F'` = fast).

Exemplo de script/query

```
conn filial1/filial1

var x number;
begin
    dbms_job.submit( :x,
                    'dbms_snapshot.refresh(''PEDIDO'', ''F'');',
                    SYSDATE + 1/1440,          -- primeira execução em 1 minuto
                    'SYSDATE + 3/1440' );      -- repete a cada 3 minutos
end;
/

-- consultar os jobs programados do usuário atual
select job, what, interval, next_date, next_sec,
       last_date, last_sec, failures
from user_jobs
```

```
order by what;

-- remover um job pelo número
exec dbms_job.remove(21);

-- consultar jobs em execução no momento
select * from dba_jobs_running;
```

Dica de prova: a aritmética de datas é em **dias**, então $1/1440$ é 1 minuto (1440 minutos em um dia) e $3/1440$ é 3 minutos. O refresh 'F' (fast) traz só as alterações; 'C' (complete) recria o snapshot inteiro.

5. SQL*Loader

Pra que serve

O `sqlldr` (SQL*Loader) é o utilitário do Oracle para **carga em massa** de dados a partir de arquivos texto externos para tabelas do banco. É a ferramenta típica de integração/ETL quando os dados chegam de um sistema externo em formato de arquivo.

Arquivos envolvidos no processo:

- `carga.ctl` -> arquivo de controle: define as colunas a importar e a delimitação dos campos lidos.
- `dados.txt` -> arquivo de interface com os dados a serem processados.
- `carga.log` -> log gerado após o processamento.
- `carga.bad` -> registros rejeitados por erro.
- `carga.dis` -> registros descartados por não atenderem a uma condição `WHEN`.

Situação hipotética

Uma cancela automática lê placas de veículos por reconhecimento de imagem e, quando a placa é de um veículo autorizado, abre e registra os dados da passagem em um arquivo texto (`dados.txt`). O banco precisa importar esses registros em massa. O `carga.ctl` descreve o layout do arquivo e em qual tabela inserir.

Exemplo de script/query

Comando de execução (tudo na mesma linha):

```
sqlldr control=carga.ctl userid=pos/pos data=dados.txt \
```

```
log=carga.log bad=carga.bad discard=carga.dis
```

Exemplo ilustrativo de um arquivo de controle `carga.ctl` (o conteúdo exato depende do layout dos dados):

```
LOAD DATA
INFILE 'dados.txt'
INTO TABLE passagens
FIELDS TERMINATED BY ';'
( placa,
  data_hora,
  autorizado )
```

Dica de prova: após executar, sempre verificar `carga.log` (sucesso/erros), `carga.bad` (rejeitados) e `carga.dis` (descartados) para avaliar a necessidade de reprocessamento.

6. Shell Script na Administração de BD

Pra que serve

Automatizar tarefas diárias de DBA por meio de scripts shell (no Linux, `bash`). As necessidades típicas são:

1. **Backup frio** do banco (cold backup, feito com o banco fora do ar). O contraponto é o **backup quente** (hot backup, com o banco no ar e em funcionamento).
2. **Monitorar o crescimento** da base de dados (datafiles do Oracle).
3. Realizar **exports** do banco (via `exp`, gerando um arquivo `.DMP` único) como complemento à política de backup.
4. **Agendar** scripts na `crontab` para execução periódica.

Qualquer log gerado pode ser enviado por e-mail ao responsável (ex.: utilitário `sendEmail`, que conecta a um SMTP, autentica e envia a mensagem com ou sem anexos).

Situação hipotética

É preciso monitorar o crescimento dos datafiles do Oracle de forma automática e periódica. O script de monitoramento (`Total_Datafiles.sh`) deve ser agendado na `crontab` do usuário `root` para rodar de hora em hora, gerando um log a cada execução. Se um filesystem ultrapassar um limite, um e-mail de aviso é disparado ao DBA.

Exemplo de script/query

Agendamento na crontab do root:

```
su - # entra como root (senha solicitada)
crontab -e # edita a crontab

# inserir a linha abaixo (executa o script como usuário oracle a cada
minuto):
* * * * * su - oracle /home/oracle/Total_Datafiles.sh
```

Os 5 campos do cron são, na ordem: **minuto hora dia-do-mês mês dia-da-semana**. O `*` significa "todos". É necessário `su - oracle` porque o `sqlplus` usado no script só está no PATH do usuário oracle, não do root.

Envio de e-mail de aviso (exemplo com sendEmail):

```
/sendEmail-v1.55/sendEmail \
-f remetente@dominio.com.br \
-t destinatario@dominio.com.br \
-u "Aviso!" \
-m "File System /oracle1 do Oracle Server < 30Gb !" \
-s smtp.dominio.com.br \
-xu usuario@dominio.com.br \
-xp senha
```

Dica de prova: diferença entre os processos: o **backup frio** copia os datafiles com o banco parado; o **export (exp)** extrai os dados logicamente para um arquivo `.DMP` com o banco no ar, servindo como complemento, e não substituto, da política de backup físico.

7. Tuning de Consultas SQL

Pra que serve

Otimizar a performance das consultas, principalmente analisando o **plano de acesso** escolhido pelo otimizador e criando **índices** adequados. Ponto central: nem sempre criar um índice melhora a performance. Em alguns casos, um índice pode até piorar ou manter o tempo igual, então é preciso medir antes e depois e ter metodologia.

Conceitos-chave

Otimizadores:

- **RBO** (Rule-Based Optimizer): monta o plano de acesso baseado em **regras**. Disponível até a versão 9. Mais simples para entender o uso de índices.
- **CBO** (Cost-Based Optimizer): monta o plano baseado nas **estatísticas** dos objetos. Introduzido na versão 7 e único a partir da versão 10. Para calcular o custo, considera número de leituras lógicas, uso de CPU e tráfego de rede.

O otimizador pode usar: pistas do desenvolvedor (hints), estatísticas, dicionário de dados e a cláusula `WHERE`.

autotrace (no SQL*Plus) mostra os planos de acesso e estatísticas:

```
set autotrace on           -- mostra plano + estatísticas + resultado
set autotrace off         -- desativa
set autotrace traceonly    -- plano + estatísticas, sem o resultado
set autotrace traceonly explain -- só o plano de acesso
set autotrace traceonly statistics -- só as estatísticas
show autotrace             -- configuração atual

-- forçar o otimizador dentro da sessão:
alter system set optimizer_mode = RULE;
alter system set optimizer_mode = CHOOSER;
```

Seletividade e ordem dos campos no índice: campos mais seletivos (que retornam menos linhas) são melhores candidatos a índice. Em um índice composto, a **ordem dos campos importa**: se o primeiro campo do índice não aparece como filtro na consulta, o índice **não é usado**.

Situação hipotética

A tabela `TNFS_SAIDA_HIST` (histórico de notas fiscais, com 4.204.223 linhas) está sem índices e uma consulta que soma o peso bruto por mês está lenta. Cria-se um índice e mede-se o tempo antes/depois (espera-se redução de cerca de 75%). Depois compara-se: índice nos 4 campos do `WHERE` versus índice só no campo mais seletivo (`peso_liq`), avaliando também o **tamanho** de cada índice.

Exemplo de script/query

Medir e analisar uma consulta:

```
conn pos/pos

-- garantir que nenhum índice atrapalhe a medição inicial
drop index tnfs_saida_hist_ix1;
drop index tnfs_saida_hist_ix2;
```

```

set timing on          -- mostra o tempo de execução
set autotrace on       -- mostra o plano de acesso

select trunc(dt_emis, 'MM') Emissao, sum(peso_brt) PesoBruto
from   tnfs_saida_hist
where  dt_emis between '30-NOV-02' and '30-NOV-04'
       and uf = 'RS'
       and sit_nf = 'I'
       and peso_liq > 100000
group by trunc(dt_emis, 'MM')
order by 1;

```

Criar índices (composto vs. seletivo):

```

-- índice composto sobre os 4 campos do WHERE (ocupa mais espaço)
create index tnfs_saida_hist_ix1
on tnfs_saida_hist (dt_emis, uf, sit_nf, peso_liq);

-- índice só sobre o campo mais seletivo (menor e, para esta consulta,
melhor)
create index tnfs_saida_hist_ix2
on tnfs_saida_hist (peso_liq);

-- tamanho de um índice em MB
select sum(bytes)/1024/1024 Mb
from user_segments
where segment_name = 'TNFS_SAIDA_HIST_IX1';

```

Avaliar a seletividade de um campo (quantas linhas cada filtro retorna):

```

select count(*) from tnfs_saida_hist where peso_liq > 100000;
select count(*) from tnfs_saida_hist where uf = 'RS';

```

"Destruir" um índice composto sem alterar a semântica da consulta. No índice (vlr_total, sit_nf, dt_emis), como vlr_total é o primeiro campo e não estava no filtro, o índice não era usado. Adicionar um predicado inofensivo no primeiro campo passa a permitir o uso do índice:

```

select uf, cidade, sum(vlr_total)
from   tnfs_saida_hist
where  dt_emis between '01-MAR-04' and '31-MAR-04'
       and sit_nf = 'C'

```

```

    and vlr_total > 0          -- predicado que "destrava" o índice composto
group by uf, cidade
having sum(vlr_total) > 1000000;

```

Estatísticas e histogramas (para o CBO):

```

-- verificar estatísticas e histogramas existentes
select table_name, num_rows from user_tables;
select * from user_histograms order by 1,2,3;

-- excluir todas as estatísticas do esquema POS
exec DBMS_STATS.delete_schema_stats('POS');

-- coletar estatísticas
analyze table TNFS_SAIDA compute statistics FOR ALL INDEXED COLUMNS;
analyze table TCLIENTES compute statistics;

```

Dica de prova: três ideias que costumam cair:

1. Índice composto só é usado se o **primeiro campo** dele estiver no filtro.
2. O campo **mais seletivo** (que restringe mais linhas) é o melhor candidato a índice, e gera um índice **menor** que um índice de vários campos.
3. O **CBO** depende de estatísticas atualizadas (`DBMS_STATS / analyze table`); sem elas, ele pode escolher planos ruins.

Mapa rápido de revisão

Tópico	Comando/sintaxe central	Para quê
Views	<code>create or replace view ... as select ...</code>	Abstração e segurança (acesso por coluna)
Database Links	<code>create database link ... using 'STR' + tabela@link</code>	Acessar/replicar dados de outro banco
Sinônimos	<code>create synonym apelido for objeto</code>	Ocultar localização física do dado
Flash Back	<code>select ... as of timestamp to_timestamp(...)</code>	Voltar no tempo / recuperar dados
Jobs	<code>dbms_job.submit(...), dbms_snapshot.refresh</code>	Automatizar/agendar tarefas e refresh

Tópico	Comando/sintaxe central	Para quê
SQL*Loader	sqlldr control=... data=... log=...	Carga em massa de arquivo externo
Shell Script	crontab -e , exp , cold/hot backup	Automatizar tarefas de DBA no Linux
Tuning	set autotrace on , create index , DBMS_STATS	Analisar plano de acesso e indexar